
The CFD Workload and Mesh Topology

How one fluid-dynamics problem is cut into 128 pieces, mapped onto a GPU cluster, and why the size of those pieces decides almost everything.

216 M

CELLS, REFERENCE MESH

128

GPUS, ONE RANK EACH

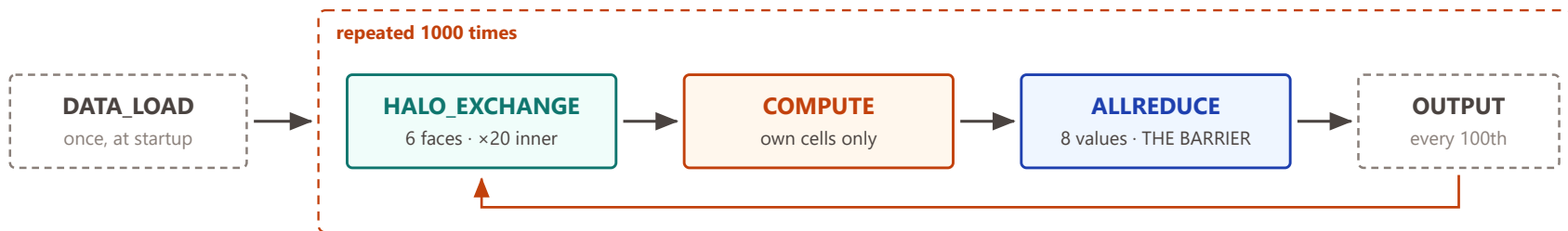
8×4×4

PROCESS GRID

THE JOB

One timestep, four phases, one barrier

A bulk-synchronous iterative solver. Every rank does the same four things, then waits for the slowest.



iteration_time = max(rank arrival times) + collective time + output time

Communication is not once per step

The halo is exchanged on every one of the **20 inner solver iterations**. That factor is what makes the network matter at all for an implicit solver.

The barrier is the measurement

ALLREDUCE moves only 64 bytes, so it is pure latency. Every rank waits for the slowest — which is how one sick GPU becomes everyone's problem.

Not a physics model

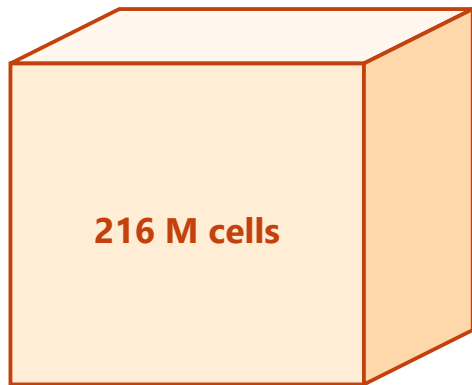
Nothing here solves the Navier–Stokes equations. The CFD framing exists to motivate **domain decomposition**, which is the actual subject.

DOMAIN DECOMPOSITION

One cube, cut into 128 subdomains

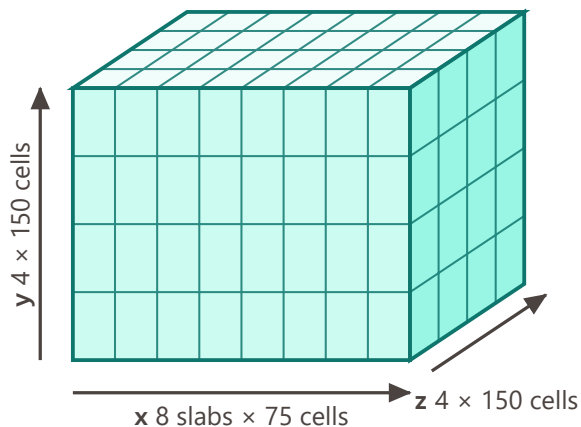
A uniform Cartesian box is split across a 3D process grid — one subdomain per rank, one rank per GPU.

The domain — 600^3 cells



partition

$8 \times 4 \times 4$ process grid



128 subdomains, each $75 \times 150 \times 150$ cells

Chosen by minimising total subdomain surface area

Of every factorisation of 128, $8 \times 4 \times 4$ gives the least halo to send.

Ties break toward the longest axis on **x** — which matters on slide 8.

6

NEIGHBOURS PER RANK, ALWAYS

The box is **triply periodic** — a Taylor–Green vortex. Every rank has six neighbours and none sits on a wall, so any imbalance in the results comes from **partitioning alone**, never from geometry.

Each rank owns

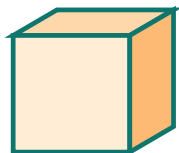
- 1,687,500 cells
- 3.4 GB of state
- 6 halo faces
- 144 MB sent / timestep

THE GOVERNING IDEA

Compute scales with volume. Communication scales with surface.

This single asymmetry explains every result that follows.

Small subdomain — coarse mesh



volume **122 k cells**

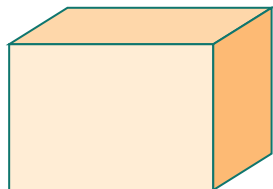
surface **15.6 k cells**

ratio 0.128

thick shell — halo is expensive

outline weight =
halo as a share
of the total work

Large subdomain — fine mesh



volume **7.81 M cells**

surface **250 k cells**

ratio 0.032

thin shell — halo nearly free

compute $\propto n^3$ (cells owned)

halo $\propto n^2$ (face cells)

ratio $\propto 1 / n$

- **Refining the mesh makes the job more efficient, not less.** Bigger subdomains hide the halo behind more work.
- **Nothing else differs between the three meshes.** Same cluster, same code, same 128 ranks — only `dims` changes.
- **Resolution alone controls the compute:communication ratio**, and with it achieved GPU utilisation. That is the entire mesh study.

THREE MESHES

The same problem at three resolutions

One cluster, one process grid, one solver. Only the cell count changes.

Mesh	Dimensions	Total cells	Cells / rank	Subdomain	Surface : volume	Memory / rank	Partition
Coarse	250^3	15.6 M	122 k	$32 \times 63 \times 63$	0.128	0.25 GB	ragged
Medium	600^3	216 M	1.69 M	$75 \times 150 \times 150$	0.053	3.4 GB	exact
Fine	1000^3	1.00 B	7.81 M	$125 \times 250 \times 250$	0.032	15.6 GB	exact

Coarse is ragged on purpose

$250 \div 8 = 31.25$ and $250 \div 4 = 62.5$ — **neither divides evenly**. Leftover cells go to the low-index ranks: some own $32 \times 63 \times 63$, others $31 \times 62 \times 62$.

127,008 max · 119,164 min · **4.0% imbalance**

Real imbalance on a **perfectly healthy cluster** — the control case for every straggler scenario.

Memory is never the constraint

Even the fine mesh uses 15.6 GB of the 80 GB on each H100-class board. The meshes are sized to isolate the **compute : communication** effect, not to stress capacity.

Halo volume, medium mesh

80 bytes per boundary cell
× 20 inner iterations
= **144 MB / rank / timestep**

5 fields × 2 cells deep × 8 bytes.

THE RESULT

Resolution decides how much of the cluster you actually get

Measured on healthy runs — no faults anywhere. Identical hardware in all three columns.

Healthy run	Coarse	Medium	Fine
Cells per rank	122 k	1.69 M	7.81 M
Mean SM occupancy	29.6%	79.1%	89.9%
Parallel efficiency	27.7%	78.0%	89.5%
Halo ÷ ideal compute	0.189	0.078	0.046
Time per timestep	39 ms	192 ms	774 ms
Mean power draw	253 W	567 W	636 W
Mean GPU temperature	39 °C	58 °C	62 °C
Reported utilisation	99.9%	99.9%	99.9%

Parallel efficiency = ideal time (perfect partition, full occupancy, zero communication) ÷ achieved time.

The coarse mesh is not network-bound

Communication is only about 6% of its timestep — much the same share as the medium mesh. It loses **roughly 70% of the machine to empty SMs**: a 122 k-cell subdomain cannot fill an H100.

$$\text{occupancy} = \text{cells} / (\text{cells} + 250,000)$$

Utilisation is a liar

All three report **99.9% GPU utilisation**. A rank spinning on a barrier keeps a kernel resident, so the device looks busy while doing nothing. Only **occupancy** tells them apart.

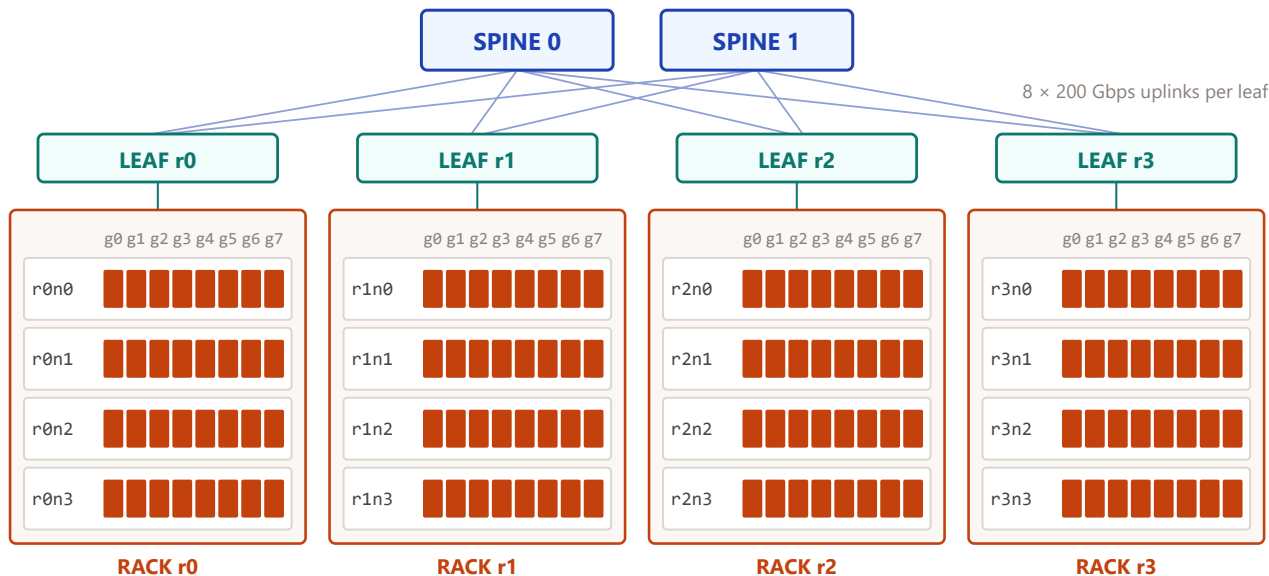
Why the fine mesh runs hot

Power follows occupancy — 253 W against 636 W. The **same** cooling fault therefore throttles the fine mesh and stays invisible on the coarse one.

THE CLUSTER

128 GPUs in a two-tier fabric

One leaf switch per rack — so a rack is simultaneously a **network domain** and a **thermal domain**.



4 racks · 16 nodes · 128 GPUs · 32 NICs · 4 leaf + 2 spine switches

Ids are rack > node > GPU: node r2n1 holds GPUs r2n1g0 ... r2n1g7, so every one of the 128 GPUs has a unique id.

One leaf switch and one cooling unit per rack — all 32 GPUs in a rack share both.

Three link classes

Path	Gbps	Latency
Inside a node NVLink-class	2400	2 μ s
Node → leaf host NIC	200	5 μ s
Leaf → spine ×8 per leaf	200	12 μ s

Latency is one-way. A cross-rack hop traverses NIC → leaf → spine → leaf → NIC: about **34 μ s**, roughly **17× the intranode cost**.

The leaf is non-blocking — 1600 Gbps up matches 1600 Gbps down — so in health the host NIC is the bottleneck and the uplink bundle runs near half capacity.

PLACEMENT

Where the mesh meets the machine

The $8 \times 4 \times 4$ grid is not arbitrary — it is chosen so the **largest halo faces ride the fastest link**.

packed placement — rank $r \rightarrow$ GPU r , with x fastest

$\pm x$ grid extent 8 $\rightarrow$ **same node**
the two largest faces · 36 MB each per timestep

NVLink

2 μ s
256 exchanges

$\pm y$ grid extent 4 $\rightarrow$ **same rack**
18 MB each per timestep

via leaf

10 μ s
256 exchanges

$\pm z$ grid extent 4 $\rightarrow$ **across racks**
18 MB each per timestep

via spine

34 μ s
256 exchanges

Counterfactual — the scatter strategy spreads adjacent ranks across racks, putting those same **36 MB $\pm x$ faces onto the 34 μ s cross-rack path** instead.

Why this is the whole trick

Row-major ordering puts x fastest, and x has grid extent 8 — exactly the GPUs in a node. So the biggest faces never leave the node.

And why faults localise

Cross-rack traffic is confined to $\pm z$. A rack-level fabric fault therefore hits exactly the $\pm z$ exchanges and leaves intra-node traffic untouched — which is what makes it diagnosable.

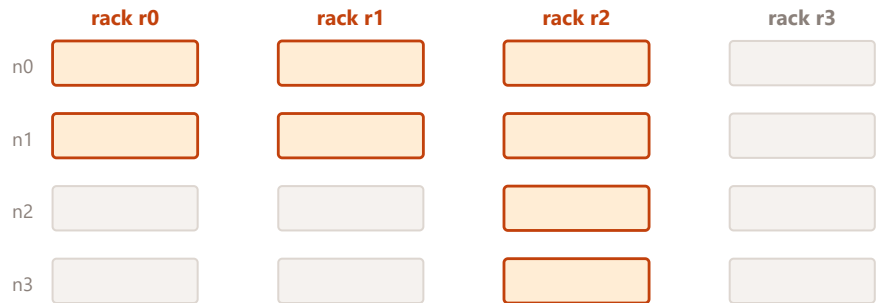
Ranks 0–7 share a node; ranks 0–31 share a rack. A rack-scoped fault appears as a **contiguous block of ranks**, never a scatter.

ALLOCATION

The job does not have to fill the cluster

128 ranks is the default, not a constraint. A job can be given any slice — and the mesh re-partitions to fit it.

64 ranks on 8 nodes — rack r3 left entirely idle



running — 64 GPUs idle — 64 GPUs, still sampled

allocation: n_ranks: 64 nodes: [r0n0, r0n1, r1n0, ...]

Ranks	Grid	Cells / rank	GB / rank	Surface : vol
128 · all GPUs	8 × 4 × 4	1.69 M	3.4	0.053
64	4 × 4 × 4	3.38 M	6.8	0.040
32	4 × 2 × 4	6.75 M	13.5	0.033

The mesh re-partitions to fit

The same 600^3 problem over 64 ranks gives each one **twice the cells** and a **better** surface-to-volume ratio. Half the machine runs the job more efficiently per GPU — and takes roughly twice as long.

The cluster never changes shape

Unallocated GPUs are not removed. They sit at **idle power and near-inlet temperature** with occupancy and utilisation pinned to zero — and they still appear in every telemetry table.

Placement then spreads over the pool

`packed` takes a prefix; `scatter` round-robins over the pool's racks. Here the gap is stark:

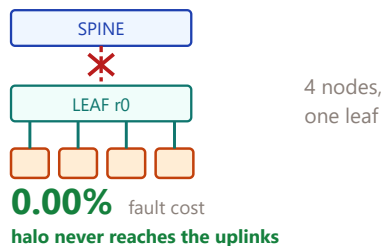
packed 192 node · 96 rack · **96 cross**
scatter 88 node · 46 rack · **250 cross**

DOES RACK LOCALITY PAY?

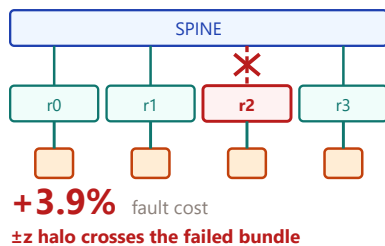
Nothing in health — everything under a fabric fault

Identical 32-rank job (4 nodes × 8 GPUs), medium mesh. Each pool run **twice**, healthy and `network_domain`, so the fault cost is a paired difference on the same GPUs.

A — job inside one rack



B — job across four racks



Node pool	Healthy	Faulted	Fault cost	Halo before → after
same rack r0	663.73	663.73	0.00%	23.1 → 23.1
same rack r2	662.13	662.13	0.00%	23.1 → 23.1
spread n0	659.73	685.16	+3.85%	23.1 → 42.6
spread n1	663.78	685.96	+3.34%	23.1 → 42.6

Post-fault means, ms per timestep. In health the two layouts differ by +0.03% — far inside the 0.70% spread that pool choice alone produces through silicon variation. The injection is retargeted into a rack the job occupies, so no arm gets a harmless fault: on pool r0 it lands on r0, not r2.

One rack is immune — to the bit

The failure is on **leaf→spine uplinks**. A job living under a single leaf never sends a packet through them, so the faulted run is **identical** to the healthy one.

Spread pays the whole bill

Its ±z halo crosses the spine through the degraded bundle — 8 uplinks down to 1, with a 2% frame error rate on the survivor. **Halo time +84%** (23.1 → 42.6 ms), iteration time +3.3 to +3.9%.

The detector fires either way

All four runs return `HARDWARE_FAULT / rack / high` — including the two that lost +0.08% and +0.13%. The counters scream while the wall clock does not, which is exactly why the fabric check runs *before* the impact gate.

The decomposition is the experiment

Every signal the simulator produces traces back to a choice on the previous slides.

Mesh size → efficiency

Subdomain volume sets SM occupancy through $\text{cells} / (\text{cells} + 250k)$. That is why coarse reaches **27.7%** parallel efficiency and fine reaches **89.5%** on identical hardware.

Occupancy then sets power, power sets temperature, and temperature sets throttling. **The workload decides how severe a hardware fault looks.**

Ragged partition → stragglers

The coarse mesh divides evenly on **no axis**, giving a 4.0% spread in cells per rank with nothing broken anywhere.

That is the control case: it shows barrier-pacing can appear on a **perfectly healthy cluster**, so any detector has to clear that bar before claiming a fault.

Placement → localisation

Because a rack is both a network and a thermal domain, and because $\pm z$ is the only cross-rack traffic, rack-scoped faults land on a **known, contiguous set of ranks**.

Without that structure a fabric fault and a cooling fault would be far harder to tell apart in the telemetry.

The one-line version. Compute grows as the cube of subdomain size and communication as the square — so the mesh you choose, not the hardware you buy, decides how much of the cluster you actually get to use.